

Table of Contents

Unit 3 — Introduction to Python Programming	2
Introduction	2
3.1 Introduction to Python Programming	3
3.1.1 Basic Programming Concepts	4
Setting Up Your Python Development Environment	4
3.2 Basic Python Syntax and Structure	6
Python Comments	7
Pakistan's national game	8
Pakistan's national flower	9
This comment is about the national animal	10
Markhor is Pakistan's national animal	11
3.2.1 Variables in Python	11
Variable Naming Rules	12
Types of Variables (Data Types)	12
Input and Output Operations	13
Handling Number Inputs	13
3.3 Operators and Expressions	14
3.3.1 Arithmetic Operators	15
3.3.2 Comparison Operators	16
3.3.3 Assignment Operators	18
3.3.4 Logical Operators	20
Truth Tables — How Logical Operators Work	20
Practical Example — Combining Comparisons with Logic	21
3.3.5 Operator Precedence in Python	22
Step-by-Step Trace: How Python Evaluates an Expression	22
Using Parentheses to Change the Order	23
Full Precedence Demonstration	23
Chapter Summary	25
Exercise Questions	25
Multiple Choice	25
Short Answer Questions	26
Long Answer / Practical Questions	26

Unit 3 — Introduction to Python Programming

A complete beginner's journey — from zero code experience to writing your own Python programs.

Student Learning Outcomes — By the End of This Chapter You Will Be Able To:

- **Understand** basic programming concepts and set up a Python development environment.
 - **Write and interpret** basic Python syntax and structure, including variables, data types, and input/output operations.
 - **Use** various operators and expressions in Python — arithmetic, comparison, assignment, logical operators, and operator precedence.
-

Introduction

STORY MODE

In December 1989, a programmer in the Netherlands named **Guido van Rossum** was bored during the Christmas holiday. He decided to spend his free time building a programming language — something simpler and more readable than what existed at the time. He named it **Python**, not after the snake, but after his favourite TV show: *Monty Python's Flying Circus*.

He had no idea that this holiday project would one day become one of the most popular programming languages on the planet — used by NASA scientists, Netflix engineers, and now — you.

Welcome. You are about to write your first Python program. And that sentence is more exciting than it sounds, because programming is genuinely one of the most powerful skills a human being can have in the 21st century.

Think about every app on your phone. Every website you visit. Every game you play. Every YouTube recommendation algorithm that somehow already knows what you want to watch next. All of it — at its core — is someone giving instructions to a computer in a language the computer can understand.

Python is that language. And you are going to learn it, step by step, starting right now.

This chapter covers everything you need to go from zero — no code experience, no technical background — to writing real, working Python programs. Take your time. Don't skip the "Pause & Think" sections. Try every code example yourself. The only way to learn programming is to actually do it.

Let's begin.

3.1 Introduction to Python Programming

Python is a **high-level programming language**. Let's break that phrase down immediately, because it contains two terms worth understanding.

high-level — This means Python is designed for humans — not machines. It uses words from the English language (like `print`, `input`, `if`, `while`) instead of the 0s and 1s that computers actually understand. Python translates your human-readable code into machine instructions automatically.

programming language — A set of rules and symbols that let you write instructions for a computer to follow. Just like Urdu or English is a language for communication between people, Python is a language for communication between a person and a computer.

Python is **versatile** — meaning it works for many different types of tasks. Developers use Python to build websites and web apps, to analyze large amounts of data, to create artificial intelligence models, to automate boring repetitive tasks, and to control robots. You will begin with the fundamentals, and those fundamentals open every single one of those doors.

Python's biggest strength is its **readability**. Compare these two lines of code — both do the same thing (display a message):

```
# Python (readable):
print("Hello, students!")
```

```
// Java (the same thing — much more code):
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, students!");
    }
}
```

Python says: here is what you want to do — say it clearly, and let's get it done. That philosophy makes it the perfect starting point.

3.1.1 Basic Programming Concepts

STORY MODE

In the 1940s, the world's first electronic computers — enormous machines the size of entire rooms — had no keyboards, no screens, and no way to type instructions. Programmers had to **physically rewire the machine** using cables and plugboards to change what it calculated. A single new calculation could take hours of rewiring work.

Then people realized: what if the machine could store its own instructions internally? What if you could write a program — a list of steps — and the machine would just follow it? That idea changed everything. Today, you can write a program in seconds that would have taken those early engineers days to set up.

And it all comes down to one core idea: **a program is a set of clear, precise instructions for a computer to follow.**

Computer programming is the process of writing instructions that tell a computer how to perform a task. The computer does exactly what you tell it — no more, no less. It does not guess. It does not assume. It follows your instructions precisely, in the exact order you wrote them.

Think of it like giving directions to a friend who has never visited your house before. You cannot say "just find it." You must say: "Turn right at the bakery. Go straight for 200 metres. Turn left at the red gate. My house is the third one on the right." The computer is like that friend — incredibly capable, but it needs exact, step-by-step guidance.

A **program** is that set of steps, written in a programming language. When Python reads your program and executes it, we say Python is **interpreting** your code.

interpreter — The program that reads your Python code and translates it into actions the computer can perform — one line at a time. When you "run" your Python program, the interpreter does this translation automatically.

Setting Up Your Python Development Environment

Before you write any code, you need to prepare your computer. This preparation is called setting up your **development environment** — it means installing the tools that let you write, run, and fix Python programs.

There are two ways to get started:

Option A — Install Python on your computer. This gives you full control and works without an internet connection.

Step-by-Step: Installing Python on Windows

1. **Open your browser and go to python.org.** At the top of the site, click on the **Downloads** section. You will see a large button showing the latest version (e.g., Python 3.13). Click it.

What happened: Your browser will download an installer file ending in `.exe`.

2. **Run the installer.** Find the downloaded file (usually in your Downloads folder) and double-click it to open it.

3. **Check the box that says "Add Python to PATH."** This is very important. You will see this option at the bottom of the first screen. If you skip this, Python may not work from your command prompt later.

What happened: This setting tells Windows where to find Python when you run it from the command line.

4. **Click "Install Now"** and wait for the installation to finish. When done, click "Close."

5. **Download an IDE.** An IDE (Integrated Development Environment) is a program where you write and run your code. Think of it as a Word Processor — but for code instead of essays. Popular choices are:

- **VS Code** — free, lightweight, widely used (recommended for beginners)
- **PyCharm** — powerful, designed specifically for Python
- **IDLE** — simple, included automatically when you install Python

6. **Test your installation.** Open your IDE, create a new file called `test.py`, and type:

```
python
```

```
print("Hello, Python!")
```

Click the Run button (or press F5 in VS Code).

What happened: If you see the words `Hello, Python!` appear in the output area at the bottom, congratulations — Python is working.

Option B — Use Google Colab (no installation needed).

Google Colab is a free platform on the internet where you can write and run Python code directly in your browser — no installation, no setup. This is perfect if you are on a school computer or a device with limited storage.

Step-by-Step: Getting Started with Google Colab

1. **Open your browser and visit:** `colab.research.google.com`
2. **Sign in** with a Google/Gmail account.
3. **Click "New Notebook."** A notebook is a document where you write code in small boxes called **cells**.
4. **Click inside a cell**, type your Python code, then press **Shift + Enter** to run it.

What happened: The output appears directly below the cell. Python is already installed in the cloud — no setup required from you.

Did you know? Google Colab comes with popular Python libraries like NumPy, Pandas, and Matplotlib already installed. These are powerful tools used in data science and artificial intelligence — and you can start using them without installing anything.

» Pause & Think

You have just set up Python (either on your computer or on Colab). Before moving on — run this one line and observe what happens:

```
python  
print("My name is [your name] and I just ran my first Python program.")
```

Replace `[your name]` with your actual name. What appeared on screen? Why do you think `print` is the right word to use here — what is it actually doing to the text you gave it?

Quick Recap: Python is a high-level, readable programming language. A program is a set of precise instructions for a computer. Before writing code, you need an environment — either Python installed locally or Google Colab in your browser. The Python interpreter reads your code and executes it line by line.

3.2 Basic Python Syntax and Structure

STORY MODE

Every language has rules about how sentences must be structured. In English, you write *"The cat sat on the mat"* — not *"Sat the on mat cat the."* The words exist, but the structure is wrong. Python works the same way. It has a **syntax** — a set of rules for how code must be written. If you follow them, Python understands you perfectly. If you break them, Python will pause and tell you something is wrong.

The best part? Python's syntax errors are not judgements. They are questions. Python is saying: *"I got a little confused here — can you clarify?"* Every programmer who has ever existed has seen a syntax error. It is part of the process.

Syntax means the rules and structure of the Python language — how code must be written for Python to understand it. Let's look at the simplest, most important piece of Python syntax: the **print statement**.

```
print("K2 is the second-highest mountain in the world.")  
print("Edhi Foundation runs the world's largest volunteer ambulance network.")
```

Output:

```
K2 is the second-highest mountain in the world.  
Edhi Foundation runs the world's largest volunteer ambulance network.
```

The `print()` function takes the text you give it — surrounded by quotation marks — and displays it on the screen. That's it. One function. One clear, simple rule.

Notice the structure:

- `print` — the function name (always lowercase)
- `()` — parentheses that contain what you want to print
- `"..."` — quotation marks around text (called a **string**)

Python also has a rule about **indentation**. Unlike most languages, Python uses the empty space at the start of a line to define the structure of your code. For now, just remember: never add random spaces at the beginning of a line.

Caution: In Python, indentation defines the structure and scope of your code. Incorrect indentation can cause errors. We will see this in action when we learn about `if` statements and loops in later chapters.

Python Comments

A **comment** is a line in your code that Python ignores completely. It is there for you — and other humans reading your code — not for the computer. Comments are used to leave notes, explanations, and reminders inside your programs.

In Python, a single-line comment starts with the `#` (hash) symbol. Everything after the `#` on that line is ignored by Python.

```
# This is a comment. Python will not run this line.  
print("This line WILL run and appear on screen.")  
  
# Comments help explain what our code does:  
print("Hello, students!") # This prints a greeting
```

Output:

```
This line WILL run and appear on screen.  
Hello, students!
```

Did you know? You can also write multiline text using triple quotes `"""like this"""` — but they are technically strings that Python reads and then ignores (because nothing is done with them). Use `#` for proper comments.

» Pause & Think

Look at the code below. Without running it, predict: how many lines will appear in the output?

```
```python
```

## Pakistan's national game

```
print("Field Hockey")
```

# Pakistan's national flower

```
print("Jasmine")
```

**This comment is about the national animal**

# Markhor is Pakistan's national animal

...

How many lines of output will appear? Why? Now run it and check if your prediction was correct.

**Quick Recap:** Syntax is the set of rules Python uses to understand your code. The `print()` function displays text on screen. Comments start with `#` and are ignored by Python — they are notes for human readers.

## 3.2.1 Variables in Python

### STORY MODE

Imagine your school bag. You put your textbook in it in the morning. After class, you take the textbook out and put your notebook in. The bag itself stays the same — what's inside changes. A variable in Python works exactly like that bag. It is a container that can hold a value, and you can change what's inside it at any time.

Every computer program works with information — a student's name, a test score, a temperature reading. Variables are how you store and keep track of that information while your program runs.

A **variable** is a named container in the computer's memory that holds a piece of data. You give the container a name, and you can put any value inside it. You can also replace that value later.

In Python, you create a variable and give it a value using the `=` sign (called the **assignment operator**):

```
Creating a variable called "age" and storing the value 71
age = 71
print("Azam lived for", age, "years")

Changing what's stored in the same variable
age = 60
print("Iqbal lived for", age, "years")
```

**Output:**

```
Azam lived for 71 years
Iqbal lived for 60 years
```

Notice: the same variable `age` held two different values at different points. That is the power of a variable — it can change. That is exactly why it is called a *variable*.

**Caution:** The `=` sign in Python does **not** mean "equals" in the mathematical sense. It means **"store this value in this variable."** Read `age = 25` as "*age gets 25*", not "*age equals 25*." This is one of the most common points of confusion for beginners — and for the record, every professional programmer was confused by this at first.

## Variable Naming Rules

You can name a variable almost anything — but Python has rules. Break these rules and Python will show an error.

Rule	Correct Example	Incorrect Example
Must start with a letter or underscore _	<code>name</code> , <code>_score</code>	<code>1name</code> , <code>2score</code>
Can contain letters, digits, underscores	<code>student_age</code> , <code>score2</code>	<code>student-age</code> , <code>score@2</code>
Case-sensitive ( <code>age</code> ≠ <code>Age</code> ≠ <code>AGE</code> )	<code>age</code> and <code>Age</code> are different	Mixing up causes bugs
Cannot be a Python keyword	<code>my_if</code> , <code>is_true</code>	<code>if</code> , <code>while</code> , <code>for</code>

**Did you know?** Python's reserved keywords — words you cannot use as variable names — include: `if`, `else`, `elif`, `for`, `while`, `break`, `continue`, `and`, `or`, `not`, `in`, `is`, `True`, `False`, `None`. These words are already claimed by Python itself.

## Types of Variables (Data Types)

Not all variables hold the same kind of data. Python automatically knows what type of data you are storing based on the value you give it.

Data Type	What It Stores	Example
int (integer)	Whole numbers — no decimal point	<code>age = 16</code>
float	Numbers with a decimal point	<code>price = 19.99</code>
str (string)	Text — always inside quotation marks	<code>name = "Ali"</code>
bool (boolean)	Only two possible values: True or False	<code>is_student = True</code>

```
Different types of variables
age = 16 # int
price = 19.99 # float
name = "Fatima" # string
is_student = True # boolean

print(name, "is", age, "years old.")
print("Is she a student?", is_student)
```

### Output:

```
Fatima is 16 years old.
Is she a student? True
```

## Input and Output Operations

A program that shows the same thing every time it runs is not very useful. Real programs interact with the person using them. Python has two key functions for this interaction:

**input()** — Displays a message, then **waits** for the user to type something and press Enter. Whatever the user types is captured and stored in a variable.

**print()** — Displays one or more values on the screen. This is Python talking back to the user.

```
Ask the user for their name, then greet them
name = input("Enter your name: ")
print("Hello, " + name + "!")
```

### Example Run:

```
Enter your name: Maryam Ashraf
Hello, Maryam Ashraf!
```

Notice the **+** sign between the strings. In Python, **+** between two text values **joins them together**. This is called **string concatenation**.

## Handling Number Inputs

Here is something important: **input()** **always gives you text** — even if the user types a number. If you need to do math with the input, you must convert it first.

```
Convert input to an integer
user_age = int(input("Enter your age: "))
print("Your age is:", user_age)

Convert input to a float (decimal number)
user_height = float(input("Enter your height in metres: "))
print("Your height is", user_height, "metres")
```

### Example Run:

```
Enter your age: 16
Your age is: 16
Enter your height in metres: 1.65
Your height is 1.65 metres
```

### » Pause & Think

You are storing a student's age in a variable called `age`. Halfway through the school year, the student has a birthday. Do you need to create a new variable called `new_age`, or can you reuse the same variable `age` and change its value?

What does your answer tell you about the word "variable" — why is it called that and not "constant"?

**Quick Recap:** A variable is a named container that stores a value. Python has four basic data types: int, float, str, and bool. Use `input()` to receive data from the user and `print()` to display data. Always convert numeric input using `int()` or `float()` when you need to do math with it.

## 3.3 Operators and Expressions

A calculator has buttons: `+`, `-`, `×`, `÷`. Each button is an **operator** — a symbol that performs an operation. Python has those same operators, plus many more powerful ones.

**operator** — A symbol that tells Python to perform a specific operation on one or more values. Examples: `+` adds, `*` multiplies, `>` compares.

**expression** — A combination of values, variables, and operators that produces a result.

`5 + 3` is an expression. So is `age > 18`. Python evaluates (calculates) an expression and gives you the result.

Python has five families of operators. Let's learn them one by one.

### 3.3.1 Arithmetic Operators

#### STORY MODE

The first electronic calculator was built in 1961 and cost the equivalent of thousands of dollars. It could only add, subtract, multiply, and divide. Today, your Python program can perform all of those operations — plus a few more — in microseconds, for free, with two lines of code. Those 1961 engineers would be astounded.

Arithmetic operators perform mathematical calculations. Here is the complete set:

Operator	Name	What It Does	Example	Result
+	Addition	Adds two numbers	<code>10 + 3</code>	<code>13</code>
-	Subtraction	Subtracts second from first	<code>10 - 3</code>	<code>7</code>
*	Multiplication	Multiplies two numbers	<code>10 * 3</code>	<code>30</code>
/	Division	Divides — always returns a float	<code>10 / 3</code>	<code>3.333...</code>
//	Floor Division	Divides — discards the decimal	<code>10 // 3</code>	<code>3</code>
%	Modulus	Returns the remainder after division	<code>10 % 3</code>	<code>1</code>
**	Exponentiation	Raises to a power	<code>10 ** 3</code>	<code>1000</code>

The three that might be new to you are `//`, `%`, and `**`. Let's look at them:

**Floor division (`//`)** — When  $10 \div 3 = 3.33$ , floor division just gives you the whole number part: 3. Imagine cutting a pizza into 10 slices and sharing them equally among 3 friends. Each friend gets 3 slices, with 1 left over.

**Modulus (`%`)** — This gives you the *remainder* of a division. `10 % 3 = 1` because 10 divided by 3 is 3 with 1 remaining. Modulus is incredibly useful — for example, to check if a number is even (`number % 2 == 0`).

**Exponentiation (`**`)** — This is "to the power of." `2 ** 10` means  $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1024$ . No need to multiply ten times by hand.

```
All arithmetic operators in action
a = 10
b = 3

print("Addition:", a + b) # 13
print("Subtraction:", a - b) # 7
print("Multiplication:", a * b) # 30
print("Division:", a / b) # 3.3333333333333335
print("Floor Division:", a // b) # 3
print("Modulus:", a % b) # 1
print("Exponentiation:", a ** b) # 1000
```

### Output:

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponentiation: 1000
```

### » Pause & Think

A shopkeeper has 47 candies and wants to put them equally into bags of 5. How many full bags will they have? How many candies will be left over? Write two Python lines — one using `//` and one using `%` — to answer both questions without doing any mental math yourself.

**Quick Recap:** Python has 7 arithmetic operators: `+`, `-`, `*`, `/` (true division), `//` (floor division), `%` (remainder), and `**` (power). Regular division always returns a float. Floor division and modulus work together to describe division completely.

## 3.3.2 Comparison Operators

### STORY MODE

Every day, your brain makes dozens of comparisons. Is it hot or cold today? Is this price higher or lower than my budget? Is this road faster or slower than the other? Comparison is how we make decisions. In Python, comparison operators do the same thing — they compare two values and answer a simple question: is this statement True or False?

**Comparison operators** compare two values and return a **Boolean result** — either `True` or `False`. There are six comparison operators:

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>10 == 10</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>10 != 5</code>	<code>True</code>
<code>&gt;</code>	Greater than	<code>10 &gt; 5</code>	<code>True</code>
<code>&lt;</code>	Less than	<code>10 &lt; 5</code>	<code>False</code>
<code>&gt;=</code>	Greater than or equal to	<code>10 &gt;= 10</code>	<code>True</code>
<code>&lt;=</code>	Less than or equal to	<code>10 &lt;= 5</code>	<code>False</code>

**Caution:** The comparison operator is `==` (double equals). The assignment operator is `=` (single equals). They look similar but do completely different things. `x = 10` stores 10 in x. `x == 10` asks: "Is x equal to 10?" Mixing these up is the most classic beginner mistake — and professional programmers still do it occasionally too.

```
Comparison operators in action
x = 10
y = 5

print("Is x equal to y?", x == y) # False
print("Is x not equal to y?", x != y) # True
print("Is x greater than y?", x > y) # True
print("Is x less than y?", x < y) # False
print("Is x greater than or equal to y?", x >= y) # True
print("Is x less than or equal to y?", x <= y) # False
```

**Output:**

```
Is x equal to y? False
Is x not equal to y? True
Is x greater than y? True
Is x less than y? False
Is x greater than or equal to y? True
Is x less than or equal to y? False
```

## » Pause & Think

A student needs 40 marks to pass an exam. Write a single Python line that checks whether a score of 38 is enough to pass. The result should be True or False. Which comparison operator will you use?

**Quick Recap:** Comparison operators compare two values and always return True or False. Remember: `==` means "is equal to?" while `=` means "store this value." These two are different — don't confuse them.

### 3.3.3 Assignment Operators

#### STORY MODE

Imagine you are keeping score in a cricket match. Every time your team scores a run, you add 1 to the score. You wouldn't erase the whole scoreboard and rewrite "7" after every ball. You would just add to whatever number is already there. Python's **compound assignment operators** work exactly like that — they update a variable's existing value in one short step.

You already know the basic assignment operator: `=`. It stores a value in a variable. But Python also has **compound assignment operators** that combine a mathematical operation with assignment in one step.

Operator	What It Means	Example	Same As
<code>=</code>	Assign a value	<code>a = 10</code>	Store 10 in a
<code>+=</code>	Add and assign	<code>a += 5</code>	<code>a = a + 5</code>
<code>-=</code>	Subtract and assign	<code>a -= 5</code>	<code>a = a - 5</code>
<code>*=</code>	Multiply and assign	<code>a *= 2</code>	<code>a = a * 2</code>
<code>/=</code>	Divide and assign	<code>a /= 2</code>	<code>a = a / 2</code>
<code>//=</code>	Floor divide and assign	<code>a //= 3</code>	<code>a = a // 3</code>
<code>%=</code>	Modulus and assign	<code>a %= 3</code>	<code>a = a % 3</code>
<code>**=</code>	Exponentiate and assign	<code>a **= 2</code>	<code>a = a ** 2</code>

```
Compound assignment operators – step by step
a = 10
b = 5
print("Starting value of a:", a)

a += b
print("After a += b:", a) # 10 + 5 = 15

a -= b
print("After a -= b:", a) # 15 - 5 = 10

a *= b
print("After a *= b:", a) # 10 * 5 = 50

a /= b
print("After a /= b:", a) # 50 / 5 = 10.0

a **= 2
print("After a **= 2:", a) # 10.0 ** 2 = 100.0
```

### Output:

```
Starting value of a: 10
After a += b: 15
After a -= b: 10
After a *= b: 50
After a /= b: 10.0
After a **= 2: 100.0
```

### ▮ Grab a Partner

One of you tracks a bank account. Start with `balance = 1000`. Your partner reads out these events one at a time, and you write the correct Python compound assignment line for each one:

- Deposit 500 rupees
- Pay a bill of 200 rupees
- Double the balance (bonus month!)

What is the final balance? Write and run the code to confirm.

**Quick Recap:** Compound assignment operators like `+=`, `-=`, and `*=` combine a math operation with storing the result back into the same variable. They are shorthand —

`a += 5` means exactly the same as `a = a + 5`.

### 3.3.4 Logical Operators

#### STORY MODE

You and your best friend are deciding whether to go to the cinema. You make your decision with conditions: "I can go if it is Saturday **AND** I finish my homework." Your friend says: "I can go if it is Saturday **OR** if my mother gives permission." Those words — AND, OR — are logical operators. They let you combine two conditions into one decision. Python uses them the same way.

**Logical operators** combine multiple True/False conditions to produce one final True/False result. There are three logical operators in Python:

Operator	Meaning	Result is True when...
and	Both conditions must be true	BOTH sides are True
or	At least one condition must be true	At least ONE side is True
not	Reverses the Boolean value	The original value was False

#### Truth Tables — How Logical Operators Work

x	y	x and y	x or y	not x
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
Logical operators in Python
x = True
y = False

AND: both must be True
print("x and y:", x and y) # False (one is False)
print("x and x:", x and x) # True (both are True)

OR: at least one must be True
print("x or y:", x or y) # True (x is True)
print("y or y:", y or y) # False (both are False)

NOT: flip the value
print("not x:", not x) # False (flips True → False)
print("not y:", not y) # True (flips False → True)
```

### Output:

```
x and y: False
x and x: True
x or y: True
y or y: False
not x: False
not y: True
```

### Practical Example — Combining Comparisons with Logic

```
Is a student eligible for a scholarship?
Condition: score must be 80 or above AND attendance must be above 75%

score = 85
attendance = 80

eligible = (score >= 80) and (attendance > 75)
print("Is the student eligible for scholarship?", eligible)
```

### Output:

```
Is the student eligible for scholarship? True
```

#### » Pause & Think

A school canteen has a special deal: a free juice if today is Friday **OR** if you are a prefect. Complete this scenario in Python:

```
python
is_friday = True
is_prefect = False
gets_juice = ??? # What goes here?
print("Gets free juice?", gets_juice)
```

What is the result? Now change `is_friday` to `False`. Does the result change? Why?

**Quick Recap:** Logical operators combine True/False conditions. `and` requires both sides to be True. `or` requires at least one side to be True. `not` flips a Boolean value. These operators let your programs make complex, multi-condition decisions.

### 3.3.5 Operator Precedence in Python

#### STORY MODE

Here is a question. What is the answer to  $2 + 3 \times 4$ ? If you answered 20, you calculated it left to right:  $(2 + 3) = 5$ , then  $5 \times 4 = 20$ . But the correct mathematical answer is **14** — because multiplication happens before addition. This rule — which operation to do first — is called **order of operations**. Python follows exactly the same rules as mathematics. Knowing these rules is the difference between code that gives you the right answer and code that gives you a surprise.

**Operator precedence** is the set of rules that determines which operation Python performs first when an expression contains multiple operators.

Here is Python's order — from highest priority (done first) to lowest priority (done last):

Priority	Operator(s)	Description	Example
1 (Highest)	( )	Parentheses — always first	<code>(2 + 3) * 4 = 20</code>
2	**	Exponentiation	<code>2 ** 3 = 8</code>
3	* / // %	Multiplication, Division, Modulus	<code>10 / 2 * 3 = 15.0</code>
4 (Lowest)	+ -	Addition, Subtraction	<code>3 + 2 - 1 = 4</code>

#### Step-by-Step Trace: How Python Evaluates an Expression

Let's carefully trace through this expression: `4 + 10 / 2 - 11 % 3`

```
Step 1: Division and Modulus first (same level, left to right)
10 / 2 = 5.0
11 % 3 = 2 (because 11 = 3*3 + 2, remainder is 2)

Step 2: Now the expression is: 4 + 5.0 - 2
Step 3: Addition and Subtraction left to right
4 + 5.0 = 9.0
9.0 - 2 = 7.0

result = 4 + 10 / 2 - 11 % 3
print(result) # 7.0
```

**Output:**

```
7.0
```

## Using Parentheses to Change the Order

```
Without parentheses – Python uses default precedence
print(3 + 2 * 5) # 13 (multiplication first: 2*5=10, then 3+10=13)

With parentheses – force addition first
print((3 + 2) * 5) # 25 (parentheses first: 3+2=5, then 5*5=25)

Exponentiation example
print(2 ** 3 * 4) # 32 (exponent first: 2**3=8, then 8*4=32)
print(2 ** (3 * 4)) # 4096 (parentheses: 3*4=12 first, then 2**12=4096)
```

### Output:

```
13
25
32
4096
```

**Did you know?** Exponentiation in Python is **right-associative**. That means

`2 ** 3 ** 2` is evaluated as `2 ** (3 ** 2)` = `2 ** 9` = 512 — not `(2 ** 3) ** 2` = 64. Python works from right to left for chained `**` operators. This matches standard mathematical notation.

## Full Precedence Demonstration

```
Let's trace through multiple examples
print(f"3 + 2 * 5 = {3 + 2 * 5}") # 13 (multiply first)
print(f"(3 + 2) * 5 = {(3 + 2) * 5}") # 25 (parentheses first)
print(f"15 / 3 * 2 = {15 / 3 * 2}") # 10.0 (left to right)
print(f"15 // 4 + 2 = {15 // 4 + 2}") # 5 (floor div first)
print(f"17 % 5 * 2 = {17 % 5 * 2}") # 4 (modulus first: 17%5=2, 2*2=4)
print(f"2 ** 3 + 1 = {2 ** 3 + 1}") # 9 (exponent first: 2**3=8, 8+1=9)
```

### Output:

```
3 + 2 * 5 = 13
(3 + 2) * 5 = 25
15 / 3 * 2 = 10.0
15 // 4 + 2 = 5
17 % 5 * 2 = 4
2 ** 3 + 1 = 9
```

**Did you know?** In Python, the letter `f` before a string creates an **f-string** (formatted string). It lets you embed the result of an expression directly inside text using curly braces `{}`. So `f"Answer: {2 + 3}"` automatically calculates `2 + 3` and places the result — 5 — inside the string. Very handy for showing results cleanly.

### ▮ Grab a Partner

Write the expression `2 + 3 * 4` on paper. One of you calculates it left to right, ignoring precedence rules:  $(2 + 3) = 5$ , then  $5 * 4 = 20$ . The other follows Python's rules: multiplication first,  $3 * 4 = 12$ , then  $2 + 12 = 14$ .

You get different answers. Now type `print(2 + 3 * 4)` in Python and run it. Whose answer does Python agree with? Why does operator precedence matter in real programs?

**Quick Recap:** Operator precedence determines which operations Python performs first. The order is: Parentheses → Exponentiation → Multiplication/Division/Modulus → Addition/Subtraction. When in doubt, use parentheses to make your intention crystal clear — they always take highest priority and make your code easier to read.

## Chapter Summary

Topic	Key Idea
Python Language	High-level, readable, versatile — used in web dev, data science, AI, automation
Programming	Writing precise, step-by-step instructions for a computer to follow
Development Environment	Install Python + an IDE (VS Code, PyCharm, IDLE) or use Google Colab online
print()	Displays output to the screen
input()	Receives input from the user — always returns a string
Comments (#)	Notes for humans; ignored by Python
Variables	Named containers in memory that store values — values can change
Data Types	int (whole numbers), float (decimals), str (text), bool (True/False)
Arithmetic Operators	+ - * / // % **
Comparison Operators	== != > < >= <= — always return True or False
Assignment Operators	= += -= = /= //= %= *=
Logical Operators	and or not — combine Boolean conditions
Operator Precedence	Parentheses → Exponent → Mult/Div/Mod → Add/Sub

## Exercise Questions

### Multiple Choice

- What is the output of `age = 25; print("Age:", age)`?  
(a) Age: 25 ✓ (b) 25 (c) Age (d) age
- Which symbol is used for single-line comments in Python?  
(a) `//` (b) `*` (c) `--` (d) `#` ✓
- What does the `%` operator return?  
(a) Decimal result of division (b) Floor of division (c) Remainder of division ✓ (d) Power
- Which of the following is a valid variable name?  
(a) `2name` (b) `_student` ✓ (c) `for` (d) `student-age`

5. What is the result of `2 + 3 * 4` ?  
(a) 20 (b) 14 ✓ (c) 24 (d) 9

## Short Answer Questions

1. Why are comments important in programming?
2. Discuss three basic data types in Python with examples.
3. Describe the difference between the integer and float data types.
4. What are logical operators? Name all three and give an example of each.
5. How do you get user input in Python? What function do you use?
6. What is the purpose of the `print()` statement?
7. What is operator precedence? Give one example that shows why it matters.
8. Write three rules for naming variables in Python.
9. What is the difference between `=` and `==` in Python?

## Long Answer / Practical Questions

1. What are the basic data types in Python? Explain integers, floats, strings, and Booleans with a code example for each.
2. What are arithmetic operators in Python? List all seven with a code example showing each one in use.
3. Explain the `input()` and `print()` functions. Write a program that asks the user for their name and favourite subject, then prints a personalised message.
4. Explain how comparison and logical operators can work together. Write a program that checks whether a student (with a given score and attendance percentage) qualifies for a school award.
5. **Program:** Write a Python program that asks the user for their name and prints a greeting message.
6. **Program:** Write a Python program that takes two numbers from the user and prints their sum.
7. **Program:** Write a Python program that takes five numbers from the user and calculates their average.
8. **Solve:** Evaluate the following expressions step by step and write what Python would print:
  - `10 // 3`
  - `2 + 3 * 4 - 1`
  - `2 ** 3 + 10 % 3`
  - `(4 + 6) * 2 - 5`